
Deep CNN Architectures

Chapter 1: AlexNet (Part 1)

Goal of this lesson:

By the end, you should understand:

- What AlexNet is.
- Why it was revolutionary.
- Its complete architecture.
- The role of each layer.

We will **not** go into mathematical convolution calculations yet. We will focus on intuition first.

What is AlexNet?

Definition

AlexNet is a deep Convolutional Neural Network (CNN) introduced in **2012** that demonstrated the power of deep learning for image classification.

It showed that a deep CNN trained on a large dataset using GPUs could significantly outperform traditional computer vision methods.

Think of AlexNet as the architecture that **started the modern deep learning revolution in computer vision**.

Why Was AlexNet So Famous?

Imagine the computer vision world before AlexNet.

Researchers believed:

"Handcrafted features are the future."

Then AlexNet came and said:

"No. Let the network learn the features itself."

That single idea changed computer vision.

Traditional Approach

Imagine classifying a LOFARGRAM.

Old pipeline:

LOFARGRAM

↓

Engineer extracts MFCC

↓

Engineer extracts Energy

↓

Engineer extracts Spectral Features

↓

SVM

↓

Prediction

The engineer decides the features.

AlexNet Approach

LOFARGRAM



CNN



Features Learned Automatically



Prediction

The network learns the features.

This is the biggest difference.

How Deep is AlexNet?

People often say:

AlexNet has **8 layers**.

What does that mean?

It has:

- **5 Convolutional Layers**
- **3 Fully Connected Layers**

Total:

$5 + 3 = 8$ Layers

Complete AlexNet Architecture

Let's see the whole architecture first.

Don't try to memorize it.

Input Image

|



Conv Layer 1

|



ReLU

|



Pooling

|



Conv Layer 2

|



ReLU

|

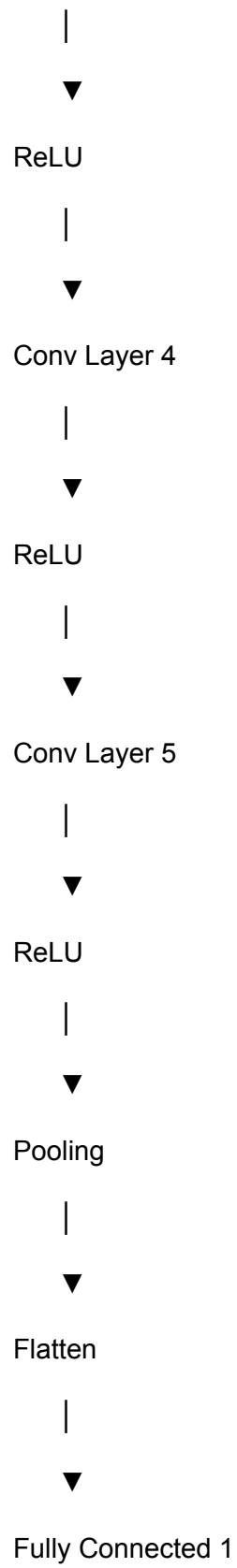


Pooling

|



Conv Layer 3



|



Dropout

|



Fully Connected 2

|



Dropout

|



Fully Connected 3

|



Softmax

|



Prediction

Don't panic.

We'll understand every single box.

Think of AlexNet Like a Factory

Imagine a factory that identifies ships.

Raw material:

LOFARGRAM

The factory has different departments.

Department 1:

Looks for simple lines.

↓

Department 2:

Looks for shapes.

↓

Department 3:

Looks for patterns.

↓

Department 4:

Makes the final decision.

Exactly the same happens inside AlexNet.

Layer 1: Input Layer

The first layer receives an image.

For your project, instead of a natural image, imagine:

LOFARGRAM

The network knows nothing yet.

It only sees numbers representing pixel intensities.

It does **not** know:

- Ship
- Ocean
- Frequency
- Engine

Just numbers.

Layer 2: First Convolution Layer

This is where learning begins.

Imagine looking at a LOFARGRAM.

Would you understand everything instantly?

No.

First, you notice:

- Bright lines
- Dark regions
- Horizontal structures

The first convolution layer does the same.

It learns **very simple features**.

Examples:

- Edges
 - Lines
 - Basic textures
-

Layer 3: ReLU

After convolution,

AlexNet applies:

ReLU

Why?

Because neural networks need **non-linearity**.

Without ReLU,

the whole network behaves like a simple linear model, no matter how many layers you stack.

ReLU allows the network to learn complex patterns.

Layer 4: Pooling

Now suppose the first layer detected:

Horizontal Line

Do we really need to remember the exact position of every pixel?

Not necessarily.

Pooling keeps the important information while reducing the size of the data.

Benefits:

- Faster computation.
 - Less memory.
 - Better robustness to small shifts.
-

Then the Process Repeats

Another convolution layer.

↓

Another ReLU.

↓

Another pooling layer.

But something changes.

What Changes in Deeper Layers?

The first layer detects:

Edges

The second layer combines those edges into:

Simple Shapes

The third layer combines shapes into:

Patterns

The fourth layer combines patterns into:

Objects

The fifth layer combines objects into:

Complete Understanding

This is called **hierarchical feature learning**.

Sonar Example

Imagine your LOFARGRAM.

Layer 1 may detect:

- Horizontal tonal lines.

Layer 2 may detect:

- Harmonic groups.

Layer 3 may detect:

- Engine frequency patterns.

Layer 4 may detect:

- Ship acoustic signatures.

Layer 5 may detect:

- "This looks like a target ship."

Notice how each layer builds on the previous one.

Fully Connected Layers

After all convolution layers,

the network has extracted many useful features.

Now it asks:

"Given all these features, what is the final answer?"

This is the job of the **Fully Connected Layers**.

They combine all learned information and make the final classification.

Softmax Layer

For AlexNet's original ImageNet task, the last layer used **Softmax** because it classified images into **1000 classes**.

Example:

Dog → 0.72

Cat → 0.10

Horse → 0.03

Car → 0.01

...

The class with the highest probability becomes the prediction.

But What About Your Project?

Your project is **binary classification**:

- Ship Present
- Ship Absent

For this type of problem, the output layer often uses **one neuron with a Sigmoid activation** (or equivalently, two outputs with Softmax in some designs).

So if you adapt AlexNet for your project, the final layer would typically be changed.

Complete Understanding

Let's summarize the journey.

LOFARGRAM

|



Conv 1

Find edges and simple lines

|



Conv 2

Combine into simple patterns



Conv 3

Find harmonic structures



Conv 4

Recognize ship-related acoustic patterns



Conv 5

Build high-level representations



Fully Connected Layers

Combine all learned information



Output

Ship Present / Ship Absent

One Important Clarification

Many beginners think:

"Each convolution layer detects ships."

That's **not true**.

Each layer learns **progressively more complex features**.

Early layers don't know what a ship is.

They only learn simple visual structures.

Only after many layers does the network learn enough information to recognize a ship.

Summary

Layer	Purpose
Input	Receives the image (LOFARGRAM)
Conv 1	Detects simple edges and lines
Conv 2	Detects simple patterns
Conv 3–5	Learns increasingly complex features
ReLU	Adds non-linearity
Pooling	Reduces size while keeping important information

Fully Connected Combines extracted features

Softmax/Sigmoid Produces the final prediction

Next Lesson: AlexNet Part 2 (Deep Dive)

In the next lesson, we won't just say "**Conv Layer 1.**"

We'll answer questions such as:

- What actually happens inside a convolution layer?
- Why are there **96 filters** in the first layer?
- Why do later layers have more filters?
- What is the shape of the data after each layer?
- How do feature maps grow and evolve?
- How does AlexNet gradually transform an image into a prediction?

That lesson will give you a much deeper understanding of how AlexNet processes data internally. After it, the architecture will no longer look like a black box.

Excellent. Now we're entering the part that most students skip.

Most people memorize:

Conv → ReLU → Pooling

But they never ask:

"What is actually happening inside a convolution layer?"

This lesson will answer that.

By the end, you'll understand **why AlexNet starts with 96 filters, why filters increase in deeper layers, and how the network slowly "understands" an image.**

AlexNet Part 2: Inside a Convolution Layer

First Question

Suppose we give AlexNet this image:

LOFARGRAM

What does the computer actually see?

Does it see:

"This is a ship."

No.

It only sees a huge matrix of numbers.

Example (very small image):

```
12 15 17 18
20 25 28 30
35 40 42 44
50 55 60 65
```

Each number is a pixel intensity.

To us:

Ship.

To the computer:

Just numbers.

Step 1: First Convolution Layer

The first convolution layer has **96 filters**.

Now think carefully.

What is one filter?

A filter is like a **small detective**.

Its job is to search for **one specific pattern**.

Imagine hiring detectives.

Detective 1:

"I'll search for horizontal lines."

Detective 2:

"I'll search for vertical edges."

Detective 3:

"I'll search for diagonal patterns."

...

Detective 96:

"I'll search for another useful pattern."

Each filter learns **one job**.

Very Important Idea

A filter is **not manually programmed**.

Nobody tells it:

"Find horizontal lines."

Instead:

Initially:

Random Numbers

During training:

Backpropagation changes those numbers.

Eventually the filter discovers:

"If I become a horizontal-line detector, the loss decreases."

The filter **learns automatically**.

This is a revolutionary idea.

What Does One Filter Produce?

Suppose Filter 1 detects horizontal lines.

Input:

LOFARGRAM

↓

Filter scans the image

↓

Output:

Feature Map 1

Now Filter 2 scans.

↓

Output:

Feature Map 2

Continue...

Filter 96

↓

One Image Becomes Many Images

This surprises many beginners.

Input:

1 Image

After Conv1:

96 Feature Maps

Think of it as 96 different "views" of the same image.

Each view highlights a different pattern.

Example

Imagine your LOFARGRAM contains:

- Horizontal engine tones
- Background noise
- Harmonics
- Broadband energy

Now imagine four filters.

Filter A highlights only:

Horizontal Lines

Filter B highlights only:

Vertical Structures

Filter C highlights only:

Strong Edges

Filter D highlights only:

Harmonics

Each filter ignores everything else.

Together they build a richer understanding of the input.

Why 96 Filters?

Students often ask:

Why exactly 96?

Could it be 50?

Yes.

Could it be 120?

Yes.

So why did AlexNet use 96?

Because the designers found it to be a good balance between:

- Learning enough features.
- Computational cost.
- GPU memory available at the time.

It's an architectural design choice—not a universal rule.

Modern networks use different numbers.

Why Do Deeper Layers Have More Filters?

AlexNet roughly increases the number of filters like this:

Conv1 → 96

Conv2 → 256

Conv3 → 384

Conv4 → 384

Conv5 → 256

At first glance, this seems strange.

Shouldn't deeper layers have fewer filters?

Actually, the opposite often happens.

Think Like a Child

Imagine teaching a child.

Step 1

Can they recognize:

- Line
- Circle

Easy.

Only a few concepts.

Step 2

Now teach:

- Face
- Car
- Tree
- House

Much harder.

More concepts are needed.

The same happens inside CNNs.

Early Layers Learn Simple Things

Examples:

- Horizontal edges
- Vertical edges
- Corners
- Curves

There aren't many basic patterns.

So fewer filters are enough.

Deeper Layers Learn Complex Things

Now imagine combining those simple features.

Examples:

- Windows
- Wheels
- Eyes
- Mouth

Or for sonar:

- Harmonic stacks
- Engine modulation patterns
- Cavitation signatures
- Propeller-related textures

There are many more complex combinations.

So deeper layers often require more filters.

Think of Building a House

A house is built from:

Bricks.

How many brick types exist?

Not many.

Now build:

- School
- Hospital
- Airport
- Stadium

The combinations become almost endless.

Simple building blocks combine into many complex structures.

CNNs work the same way.

What Happens After Conv1?

Suppose Conv1 produces:

96 Feature Maps

Does Conv2 go back to the original image?

No.

This is one of the biggest misconceptions.

Conv2 receives the **feature maps from Conv1**, not the raw image.

So the information flows like this:

Original Image

↓

Feature Maps

↓

New Feature Maps

↓

More Complex Feature Maps

Every layer builds on what the previous layer has already learned.

Your Sonar Project

Imagine this progression.

Conv1

Find:

- Horizontal tonal lines
 - Simple frequency edges
-

Conv2

Combine them into:

- Harmonic groups
-

Conv3

Combine harmonics into:

- Engine signatures

Conv4

Combine engine signatures into:

- Ship-specific acoustic patterns
-

Conv5

Recognize:

"This resembles the acoustic signature of a ship."

Notice the gradual increase in understanding.

Why Not Detect Ships in the First Layer?

Imagine asking a child:

"Recognize a Boeing 747."

Before teaching them:

- Lines
- Shapes
- Wings
- Engines

Impossible.

Recognition happens step by step.

CNNs learn in the same hierarchical manner.

The Biggest Misconception

Many beginners imagine this:

Conv1



Ship Detector

Wrong.

Instead, think of it like this:

Conv1



Lines



Conv2



Shapes



Conv3



Patterns



Conv4



Objects



Conv5

↓

Meaning

Each layer adds another level of abstraction.

Summary

Layer	Learns
Conv1	Edges, simple lines, basic textures
Conv2	Combinations of edges, simple patterns
Conv3	More complex textures and structures
Conv4	High-level feature combinations
Conv5	Task-specific representations used for classification

A Thought Exercise

Imagine I remove **Conv5**.

Would AlexNet still detect ships?

Maybe, but not as well.

Why?

Because it has lost its deepest understanding.

Now imagine I remove **Conv1**.

The whole network fails.

Why?

Because without basic features, the later layers have nothing meaningful to build upon.

This shows that **every layer depends on the previous one**.

Next Lesson (AlexNet Part 3) — The Most Important One

So far you've learned **what** the layers do.

Next, we'll learn **how data actually flows through AlexNet**.

We'll trace a single image from the input to the output and answer questions like:

- Why does the image size shrink after each stage?
- Why do the number of feature maps increase?
- What is the difference between **image height/width** and **channel depth**?
- What does a tensor shape like **$224 \times 224 \times 3$** or **$55 \times 55 \times 96$** really mean?
- How does AlexNet transform one image into a final prediction?

Once you understand that, you'll stop seeing CNNs as magic boxes and start understanding them as **information-processing pipelines**. That understanding will also make architectures like **ResNet** and **EfficientNet** much easier to grasp later.

Next

Excellent. This is probably the **most important lesson** in CNNs.

If you truly understand today's lesson, then **ResNet, EfficientNet, MobileNet, DenseNet—all will become much easier**.

Today we answer the question:

"How does one image travel through AlexNet?"

This lesson is not only about AlexNet—it's about **how every CNN processes data**.

AlexNet Part 3: Data Flow Inside the Network

First, One Common Misconception

Many beginners imagine this:

Input Image



CNN



Output

It looks like a black box.

But inside the CNN, **the image is constantly changing**.

Not the object in the image—the **representation** of the image.

Imagine You're an Intelligence Officer

Suppose someone gives you a sonar recording.

Initially, you know nothing.

Step 1:

You notice a few frequency lines.



Step 2:

You recognize harmonic patterns.



Step 3:

You recognize engine characteristics.



Step 4:

You conclude:

"This is likely a ship."

A CNN works exactly like this.

Step 1: The Input

Suppose the input image size is:

$224 \times 224 \times 3$

Let's understand each number.

What does 224 mean?

Height:

224 pixels

Width:

224 pixels

So the image has:

- Height = 224
 - Width = 224
-

What does the 3 mean?

This is **NOT** three images.

It means:

Red Channel

Green Channel

Blue Channel

A color image has three channels.

So:

$224 \times 224 \times 3$

means:

- 224 pixels high
 - 224 pixels wide
 - 3 color channels
-

But What About Your Sonar Project?

Your LOFARGRAM is usually:

$224 \times 224 \times 1$

Why?

Because it is generally treated as a **grayscale image**.

There is only one intensity value per pixel instead of separate red, green, and blue values.

What is a Tensor?

This is one of the most important deep learning terms.

Definition

A **tensor** is simply a way of organizing data.

Examples:

A single number:

7

This is a **0D tensor** (a scalar).

A list:

2 4 6

This is a **1D tensor** (a vector).

A table:

1 2

3 4

This is a **2D tensor** (a matrix).

An image:

Height

×

Width

×

Channels

This is a **3D tensor**.

During training, we process many images at once.

Example:

32 images

↓

$32 \times 224 \times 224 \times 1$

This is a **4D tensor**.

The first dimension is the **batch size**.

The Journey Begins

Input:

$224 \times 224 \times 1$

↓

Conv1

What Happens in Conv1?

Excellent! This is the **final lesson on AlexNet**, and in my opinion, it's the most important one.

After today, you won't just know **what AlexNet is**—you'll understand **why it changed deep learning** and **why it was eventually replaced**.

AlexNet Part 4: Why It Worked, Its Limitations, and Its Legacy

Before AlexNet, researchers already knew about CNNs. So the question is:

Why didn't CNNs become popular before 2012?

Because several key problems prevented them from working well in practice.

AlexNet succeeded because it solved many of those problems at the same time.

Breakthrough 1: ReLU Activation Function

Let's start with one of the biggest innovations.

Earlier neural networks often used the **Sigmoid** activation function.

Its graph looks like an S-shaped curve.

The problem?

For very large positive or negative inputs, the output changes very little.

That means the gradients become extremely small.

This is called the **Vanishing Gradient Problem**.

What is the Vanishing Gradient Problem?

Imagine you're climbing a mountain.

At first, the slope is steep, so you know which direction to go.

But suddenly the ground becomes almost flat.

Now you barely know which way is uphill.

Learning becomes extremely slow.

That's what happens in deep neural networks when gradients become tiny.

The early layers stop learning effectively.

Why ReLU Helped

The **ReLU (Rectified Linear Unit)** activation is simple:

- If the input is positive → output the same value.
- If the input is negative → output 0.

Because positive values are not squeezed into a narrow range like Sigmoid, gradients remain much larger in many cases.

Benefits:

- Faster learning.
- Simpler computation.
- Better training of deeper networks.

Even today, ReLU (or one of its variants like Leaky ReLU or GELU) is widely used.

Breakthrough 2: Dropout

Before AlexNet, overfitting was a major problem.

The model would memorize the training data.

AlexNet introduced **Dropout** in its fully connected layers.

Remember what we learned:

During training:

Neuron A → OFF

Neuron B → ON

Neuron C → OFF

Neuron D → ON

Each training step uses a slightly different network.

This forces the model to learn more robust features.

Breakthrough 3: GPU Training

Imagine you have to multiply millions of numbers.

A CPU has only a few powerful workers.

A GPU has thousands of smaller workers that can perform many operations simultaneously.

CNNs perform many matrix operations, which GPUs handle extremely well.

Without GPU acceleration, AlexNet would have taken far too long to train.

Today, almost all deep learning models are trained on GPUs (or specialized hardware like TPUs).

Breakthrough 4: Large Dataset (ImageNet)

AlexNet wasn't trained on just a few thousand images.

It was trained on **ImageNet**, which contained over a million labeled training images.

Why is this important?

Deep networks have millions of parameters.

If you train them on a tiny dataset, they tend to memorize instead of generalize.

Large datasets allow them to learn meaningful patterns.

Why AlexNet Was Better Than Traditional Machine Learning

Traditional pipeline:

Signal
↓
Handcrafted Features
↓
SVM
↓
Prediction

AlexNet:

Signal
↓
CNN learns features automatically
↓
Prediction

The biggest change wasn't just using a CNN.

It was **automatic feature learning**.

This reduced the need for manually designing features.

Strengths of AlexNet

AlexNet had several major strengths:

1. Automatic Feature Learning

It learned useful features directly from data.

2. Deep Architecture

Five convolutional layers allowed hierarchical feature extraction.

Simple features became complex representations.

3. ReLU

Training became much faster compared to older activation functions.

4. Dropout

Reduced overfitting.

5. GPU Training

Made large-scale deep learning practical.

But AlexNet Was Not Perfect

Every architecture has weaknesses.

AlexNet had several.

Limitation 1: Too Many Parameters

AlexNet had around **60 million parameters**.

That's enormous.

More parameters mean:

- More memory.
 - Longer training.
 - Higher risk of overfitting.
-

Limitation 2: Large Fully Connected Layers

The fully connected layers contained a huge fraction of the parameters.

They were computationally expensive and contributed heavily to the model size.

Later architectures tried to reduce or eliminate this dependence.

Limitation 3: Limited Depth

AlexNet had only **8 learnable layers**.

Researchers wondered:

"If 8 layers are good, will 16 or 50 layers be even better?"

The answer turned out to be **not always**.

Making networks deeper introduced new challenges.

Limitation 4: Vanishing Gradient in Deeper Networks

Even with ReLU, training very deep networks became difficult.

As researchers built deeper CNNs, gradients could still weaken as they propagated backward.

This motivated architectures like **ResNet**, which introduced **skip (residual) connections** to make training much deeper networks possible.

Why Don't We Use AlexNet Today?

Modern architectures generally achieve:

- Higher accuracy.
- Fewer parameters.
- Better computational efficiency.
- Improved training stability.

As a result, AlexNet is now mainly studied for its historical importance.

However, many of its ideas are still fundamental.

What Ideas Survived?

Even though we don't usually use AlexNet directly, its core ideas are everywhere.

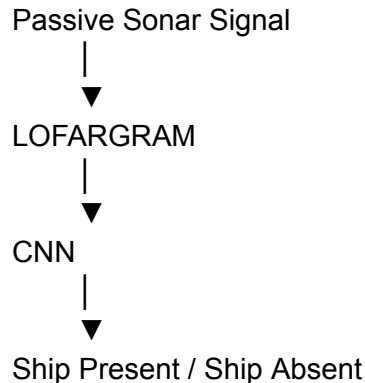
Modern CNNs still use:

- Convolutional layers.
- ReLU (or related activations).
- Pooling (though sometimes replaced or reduced).
- End-to-end feature learning.
- GPU acceleration.
- Transfer learning.

AlexNet laid the foundation that later models built upon.

How AlexNet Relates to Your Sonar Project

Imagine your pipeline:

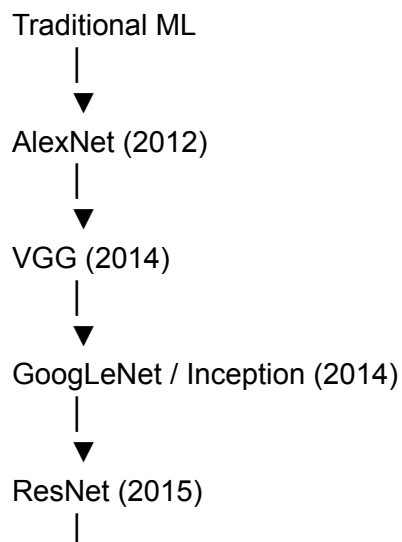


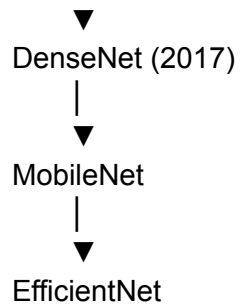
AlexNet teaches the core principle:

Don't manually design every feature. Let the CNN learn useful acoustic patterns from the LOFARGRAM.

Modern models like **ResNet-18** and **EfficientNet** follow the same philosophy—but they do it more effectively.

Timeline of CNN Evolution





Notice something important:

Each architecture was created to solve a limitation of the previous one.

Summary

Topic	Key Idea
ReLU	Faster and more effective training than Sigmoid
Dropout	Reduces overfitting
GPU Training	Makes deep learning practical
ImageNet	Provides enough data to train deep CNNs
Strengths	Automatic feature learning, hierarchical representations
Weaknesses	Large model, many parameters, difficult to scale to much deeper networks

Think Like a Researcher

If you read a paper that says:

"We used ResNet-18 instead of AlexNet."

You should immediately ask:

- What limitation of AlexNet are they trying to overcome?
- Does ResNet improve accuracy?

- Does it train deeper networks more effectively?
- Does it reduce overfitting?
- Is it more suitable for my sonar dataset?

That's the mindset that helps you critically understand research papers.

What's Next?

Now we move to **VGGNet**.

This is where the story becomes even more interesting.

Researchers asked a simple but powerful question:

"If AlexNet has 8 layers, why not make it 16 or 19 layers?"

It sounds obvious—but that decision led to both impressive improvements and entirely new problems, which eventually gave birth to **ResNet**.

VGG is the bridge between AlexNet and ResNet, so understanding it will make the next major architecture much easier to grasp.

Excellent! Welcome to the next milestone in CNN evolution.

Today, we're going to study **VGGNet**, one of the simplest yet most influential CNN architectures ever created.

Before we begin, I want you to remember one thing:

Every architecture exists because someone asked a simple question.

For AlexNet, the question was:

"Can a deep CNN outperform handcrafted features?"

For VGG, the question was:

"What if we make the network much deeper?"

That simple question changed CNN research.

Deep CNN Architectures

Chapter 2: VGGNet (Part 1)

The Story Begins

After AlexNet won the ImageNet competition in 2012, researchers were amazed.

Everyone thought:

"AlexNet has 8 layers and works great."

Then researchers at the **Visual Geometry Group** asked:

"Instead of designing a complicated architecture, what if we simply stack more convolution layers?"

That idea became **VGGNet**.

What is VGG?

VGG stands for:

Visual Geometry Group

It is the name of the research group at the University of Oxford that developed the architecture.

So:

- VGG is **not** a technical term.
 - It is the name of the research lab.
-

What Was the Main Goal?

AlexNet looked something like this:

Conv



Pooling



Conv



Pooling



Conv



FC

Researchers wondered:

"Can deeper networks learn richer and more complex features?"

The answer was:

Yes... but with some new challenges.

The Philosophy of VGG

AlexNet tried many ideas at once.

VGG followed a different philosophy:

Keep the architecture as simple as possible.

Instead of using different filter sizes everywhere,

they asked:

"What if every convolution uses exactly the same small filter?"

This made the architecture elegant and easy to understand.

The Biggest Idea

VGG made one bold decision:

Use only 3×3 convolution filters almost everywhere.

This became one of the most influential design choices in CNN history.

Wait...

You might ask:

"Why use a tiny 3×3 filter?"

Wouldn't a larger filter like 7×7 see more information?

Excellent question.

Let's understand it.

Imagine Reading a Book

Suppose you're reading a novel.

Option A:

Read **7 pages** at once.

Hard.

Too much information.

Option B:

Read **3 pages**.

Understand them.

Then read the next **3 pages**.

Much easier.

CNNs work similarly.

Instead of one huge filter,

they use several small filters.

Large Filter vs Small Filters

Imagine we want to understand this pattern.

Option 1:

7×7 Filter

One very large filter.

Option 2:

3×3

↓

3×3

↓

3×3

Three small filters.

Both approaches can eventually capture large patterns.

But the second approach has important advantages.

Advantage 1: More Non-Linearity

Remember ReLU?

After every convolution,

we apply:

Conv

↓

ReLU

↓

Conv

↓

ReLU

↓

Conv

↓

ReLU

Notice something interesting.

Three small convolutions give us **three ReLU activations**.

One large convolution gives us only **one ReLU**.

More ReLU layers mean the network can learn **more complex relationships**.

Advantage 2: Fewer Parameters

This is one of the smartest ideas in VGG.

Let's compare.

Suppose:

Input has one channel.

Using one:

7×7

filter requires:

49 weights

because:

$$7 \times 7 = 49$$

Now use three:

3×3

filters.

Each has:

9 weights

Three filters:

$$9 + 9 + 9 = 27 \text{ weights}$$

Compare:

Large filter:

49

Small filters:

Much fewer parameters.

(In real networks with many input and output channels, the exact parameter count depends on those channels too, but the same intuition generally holds: stacking smaller filters can be more parameter-efficient than using one very large filter.)

Why Fewer Parameters Matter

Fewer parameters mean:

- Less memory.
- Faster computation.
- Lower risk of overfitting.

So VGG achieved similar expressive power while using smaller filters repeatedly.

Building Features Step by Step

Imagine your sonar project.

One large filter tries to learn:

Entire Ship Signature

in one step.

Very difficult.

Instead:

First 3×3 filter:

Learns:

- Small frequency edges.

↓

Second filter:

Learns:

- Harmonic groups.

↓

Third filter:

Learns:

- Engine acoustic patterns.

This gradual learning is much easier.

VGG Became Deeper

AlexNet:

8 Layers

VGG:

Two famous versions:

VGG16

and

VGG19

The numbers indicate the number of **learnable layers** (convolution + fully connected layers).

So:

VGG16 has **16 learnable layers**.

VGG19 has **19 learnable layers**.

Compared to AlexNet,

this was a huge increase in depth.

Simple Architecture

A simplified view of VGG16 looks like:

Input



Conv



Conv



Pooling



Conv



Conv



Pooling



Conv



Conv



Conv



Pooling



Conv



Conv



Conv



Pooling



Conv



Conv



Conv



Pooling



Fully Connected



Fully Connected

↓

Fully Connected

↓

Output

Notice the pattern.

Instead of designing different kinds of blocks,

VGG simply repeats the same idea:

Small convolutions followed by pooling.

This simplicity made it very influential.

Why Researchers Loved VGG

Imagine you're building a house.

AlexNet says:

Use:

- Large bricks.
- Small bricks.
- Different shapes.

VGG says:

No.

Use only one brick size.

Repeat it.

Construction becomes much simpler.

That's exactly what VGG did.

Sonar Analogy

Imagine your LOFARGRAM.

Instead of detecting an entire ship immediately,

the network learns:

Step 1:

Small frequency changes.

↓

Step 2:

Small harmonic patterns.

↓

Step 3:

Larger acoustic structures.

↓

Step 4:

Complete ship signatures.

The learning process becomes more gradual and hierarchical.

Summary

AlexNet	VGG
8 learnable layers	16 or 19 learnable layers
Mixed filter sizes	Mostly 3×3 filters

More complex design	Very simple, repetitive design
Deep for its time	Much deeper
Started the CNN revolution	Showed that deeper networks can significantly improve performance

One Question for You

Imagine you're designing a CNN for passive sonar.

Which sounds easier?

Option A:

One giant filter tries to recognize an entire ship signature.

OR

Option B:

Many small filters gradually learn:

- Frequency edges
- Harmonics
- Engine patterns
- Complete ship signatures

Hopefully you chose **Option B**.

That's the key insight behind VGG.

Next Lesson (VGG Part 2)

Now we'll go much deeper.

We'll answer questions like:

- **Why exactly 3×3?**
- **How does stacking two 3×3 convolutions compare with one 5×5 convolution?**

- Why does VGG increase the number of filters from $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$?
- How does a LOFARGRAM travel through VGG layer by layer?
- Why did making the network deeper eventually create the vanishing gradient problem?

That last question is especially important, because it leads directly to the invention of **ResNet**, which is one of the most widely used architectures in passive sonar classification research.

Excellent. This is one of the **most important lessons** in CNN architecture.

Today you're going to understand **why VGG uses 3×3 filters**, **why the number of filters increases**, and **how the data flows through the entire VGG network**.

By the end of this lesson, you'll start thinking like the engineers who designed VGG.

Deep CNN Architectures

Chapter 2: VGGNet (Part 2)

First Question

Suppose I give you two choices.

Option A

One convolution:

5×5 Filter

Option B

Two convolutions:

3×3

↓

3×3

Which is better?

Most beginners say:

" 5×5 is better because it's larger."

But surprisingly...

Two 3×3 convolutions are usually better.

Let's understand why.

Imagine Looking at a Ship

Suppose you're trying to identify a ship.

Option A:

You look at the **entire ship at once**.

It is difficult to understand every detail.

Option B:

First, you observe:

- The bow
- The deck
- The engine area

Then you combine those observations.

This is much easier.

Deep networks work the same way.

Why Two 3×3 Filters Work Better

Let's compare step by step.

Method 1

Input



5×5 Conv



Output

One learning step.

Method 2

Input



3×3 Conv



ReLU



3×3 Conv



Output

Now something important happens.

You get:

- Two convolution operations.
- **Two opportunities to learn.**
- **One extra ReLU** between them.

That extra ReLU allows the network to learn more complex, non-linear patterns.

Think of Learning Mathematics

Suppose I ask:

Solve Differential Equations.

Without knowing:

- Addition
- Algebra
- Calculus

Impossible.

Instead:

Learn one topic.

↓

Practice.

↓

Learn the next topic.

↓

Practice.

Small learning steps are more effective.

That's exactly what VGG does.

Why Does VGG Increase the Number of Filters?

This is one of the most misunderstood topics.

Look at the progression:

64

↓

128

↓

256

↓

512

↓

512

Students often ask:

"Why keep increasing the filters?"

Let's understand.

Think About School

When you're in first grade,

you only learn:

- A
- B
- C

Very few concepts.

In college,

you study:

- AI
- Deep Learning
- Operating Systems
- Computer Networks
- Compiler Design

Many more concepts.

The deeper you go,

the **more knowledge** you need.

CNNs behave similarly.

Early Layers

Early layers detect only:

- Edges
- Lines
- Corners

There aren't many basic visual patterns.

So **64 filters** are enough.

Middle Layers

Now the network starts combining those features into:

- Curves

- Textures
- Harmonic groups

There are many more possible combinations.

So the network needs more filters.

Deep Layers

Now it begins recognizing:

For natural images:

- Eyes
- Wheels
- Windows
- Fur

For your sonar project:

- Harmonic stacks
- Engine modulation
- Propeller signatures
- Tonal structures
- Ship acoustic fingerprints

These are much more diverse.

So **512 filters** make sense.

Data Flow Through VGG16

Let's walk through the entire network.

Input

Imagine your LOFARGRAM.

$224 \times 224 \times 1$

Block 1

Conv

↓

Conv

↓

Pooling

Output:

Approximately

$112 \times 112 \times 64$

Notice:

- Height and width become smaller.
 - Channels increase.
-

Block 2

Again:

Conv

↓

Conv

↓

Pooling

Output:

$56 \times 56 \times 128$

Now the network knows much more than before.

Block 3

Conv

↓

Conv

↓

Conv

↓

Pooling

Output:

$28 \times 28 \times 256$

Notice something.

The network became deeper.

Three convolution layers before pooling.

This allows it to learn more detailed patterns before reducing the spatial resolution.

Block 4

Again:

Conv

↓

Conv

↓

Conv

↓

Pooling

Output:

$14 \times 14 \times 512$

Now the network is learning high-level representations.

Block 5

Again:

Conv

↓

Conv

↓

Conv

↓

Pooling

Output:

$7 \times 7 \times 512$

At this point, the original image has been transformed into a compact but information-rich representation.

Then Comes Flatten

The tensor:

$$7 \times 7 \times 512$$

is flattened into one long feature vector.

Then it passes through the fully connected layers for classification.

Why Does Height Decrease While Channels Increase?

Imagine reading a novel.

At the beginning,

you notice every word.

As you continue,

you stop focusing on individual words.

Instead,

you understand:

- Sentences.
- Paragraphs.
- The story.

Details decrease.

Understanding increases.

That's exactly what happens in CNNs.

Sonar Analogy

Suppose Conv1 detects:

- Horizontal frequency lines.

Conv2 combines them into:

- Harmonic groups.

Conv3 combines harmonics into:

- Engine characteristics.

Conv4 combines engines into:

- Ship acoustic signatures.

Conv5 finally understands:

"This recording likely contains a ship."

The network doesn't jump directly to the answer.

It builds understanding layer by layer.

Why VGG Performed So Well

Researchers discovered something surprising.

A **deeper network** could learn richer feature hierarchies than a shallower one.

VGG showed that simply increasing depth—while keeping the architecture simple—could significantly improve image classification performance.

This influenced many later CNN architectures.

But...

Something unexpected happened.

Researchers tried:

20 Layers

↓

30 Layers

↓

40 Layers

↓

50 Layers

They expected accuracy to keep improving.

Instead...

Training became difficult.

Sometimes a deeper network performed **worse** than a shallower one.

Why?

This wasn't just overfitting.

The main issue was that very deep networks became hard to optimize because gradients had difficulty flowing backward through many layers.

This is the **vanishing gradient problem**, and it became a major obstacle.

Researchers needed a new idea.

That idea became **ResNet**.

Summary Table

Stage	What Happens
Early Layers	Learn edges and simple structures
Middle Layers	Learn textures and combinations
Deep Layers	Learn high-level concepts such as ship signatures
Height & Width	Gradually decrease
Channels	Gradually increase
Multiple 3×3 Convs	More learning capacity with additional non-linearity

A Critical Question

Imagine you build a **100-layer CNN**.

Do you think it will always outperform a **16-layer CNN**?

Most beginners answer:

"Yes."

But the surprising answer is:

Not necessarily.

Without the right architecture, a very deep network can actually become **harder to train** and perform worse.

That surprising discovery led directly to the invention of **ResNet**, one of the most important architectures in deep learning—and one of the most widely used models in passive sonar classification research.

Next Lesson (VGG Part 3 – Final)

In the final VGG lesson, we'll answer:

- Why did VGG become so popular?
- Why is VGG rarely used in modern research despite its success?
- What are its biggest limitations?
- Why does increasing depth eventually fail?
- How did VGG inspire **ResNet**, **DenseNet**, and **EfficientNet**?

After that, we'll begin **ResNet**, which is arguably the **single most important architecture** for your passive sonar project because many sonar classification papers use **ResNet-18** or its variants as a backbone.

Next

Excellent! This is the **final lesson on VGG**, and it's one of the most important because it explains **why VGG was revolutionary, why it eventually became outdated, and how it directly led to ResNet**.

Deep CNN Architectures

Chapter 2: VGGNet (Part 3 – Final)

Goal of this lesson:

By the end, you will understand:

- Why VGG became so popular.
- Why researchers stopped using it as often.
- Why making networks deeper eventually failed.
- Why ResNet was invented.

First Question

Imagine it's 2014.

You have two CNNs.

AlexNet

- 8 learnable layers
- Mixed filter sizes
- More complex architecture

VGG16

- 16 learnable layers
- Simple repeated design
- Only 3×3 convolutions (mostly)

Which one would be easier to understand and modify?

Obviously:

VGG.

Its simplicity was one of its greatest strengths.

Why Did Researchers Love VGG?

VGG proved something very important:

Increasing depth, while keeping the architecture simple, can significantly improve performance.

This was a major insight.

Before VGG, many people focused on designing increasingly complicated convolution blocks.

VGG showed that **a consistent, repetitive design could work extremely well.**

Why Was VGG Easy to Understand?

Think of building with LEGO.

Imagine every brick is the same size.

Construction becomes straightforward.

That's VGG.

Every block follows the same pattern:

Conv 3×3

↓

ReLU

↓

Conv 3×3

↓

ReLU

↓

Pooling

The same idea is repeated throughout the network.

This simplicity made VGG a favorite for teaching and experimentation.

Why Was VGG So Accurate?

Let's compare feature learning.

AlexNet:

Edges

↓

Shapes



Objects

VGG:

Edges



Simple Shapes



Textures



Patterns



Object Parts



Objects

Because VGG was deeper, it could learn a **richer hierarchy of features**.

For your sonar project, that hierarchy might look like:

Frequency edges



Harmonic lines



Engine modulation



Propeller signatures



Complete ship acoustic signature

Each layer refines the understanding.

But Then Problems Appeared...

Researchers thought:

"If 16 layers are good...

Then 32 should be better...

64 should be even better...

100 must be amazing!"

So they built deeper CNNs.

What happened?

Surprisingly...

Sometimes the deeper networks performed **worse**.

Wait...

Isn't More Layers Always Better?

Intuitively:

More layers should mean:

- More learning.

- Better features.
- Higher accuracy.

But in practice...

That didn't always happen.

Why?

The Optimization Problem

Imagine you're passing a message through a chain of 100 people.

Person 1 says:

"Ship detected."

By the time it reaches person 100:

The message might become:

"Something... detected... maybe..."

Information gets degraded.

Something similar happens during training.

The learning signal (the gradient) has to travel backward through every layer.

As the network gets deeper:

- The signal can become very small.
- Early layers stop learning effectively.

This is one aspect of the **vanishing gradient problem**.

Another Problem: Huge Number of Parameters

VGG16 contains approximately:

138 million parameters.

That's enormous.

Let's compare:

Model	Approximate Parameters
--------------	-------------------------------

AlexNet	~60 million
---------	-------------

VGG16	~138 million
-------	--------------

VGG had more than **double** the parameters of AlexNet.

Consequences:

- Requires more memory.
- Slower training.
- Slower inference.
- Greater risk of overfitting on small datasets.

Why So Many Parameters?

The main reason is the **fully connected layers**.

The convolutional layers are relatively efficient.

But after flattening, VGG feeds a huge feature vector into three large fully connected layers.

Those layers contain the majority of the parameters.

Later architectures tried to reduce or remove this bottleneck.

Can VGG Run on Small Devices?

Imagine trying to deploy VGG16 on:

- A mobile phone.
- A small embedded device.
- An underwater robot with limited hardware.

Not ideal.

The model is large and computationally expensive.

This motivated later architectures like **MobileNet** and **EfficientNet**, which were designed to be much more efficient.

Why Is VGG Still Important Today?

Even though newer architectures often perform better, VGG remains important because:

1. It is easy to understand.

Many people learn CNNs through VGG.

2. It demonstrated the power of depth.

It showed that deeper networks could achieve better performance.

3. It is widely used for transfer learning.

Pretrained VGG models have been used as feature extractors in many applications, including medical imaging, industrial inspection, and sometimes sonar research.

Transfer Learning (A Preview)

Suppose someone has already trained VGG on millions of images.

Instead of starting from scratch, you can:

- Keep most of the learned weights.

- Replace the final classification layer.
- Fine-tune the model for your own task.

This is called **Transfer Learning**.

We'll study it in detail later because it's extremely important for your project.

The Biggest Lesson from VGG

VGG taught researchers:

"Simply stacking more layers is not enough."

Eventually, optimization becomes difficult.

Researchers needed a way to train much deeper networks.

That solution came from **ResNet**.

The Question That Changed Deep Learning

Researchers asked:

"What if we let information skip some layers?"

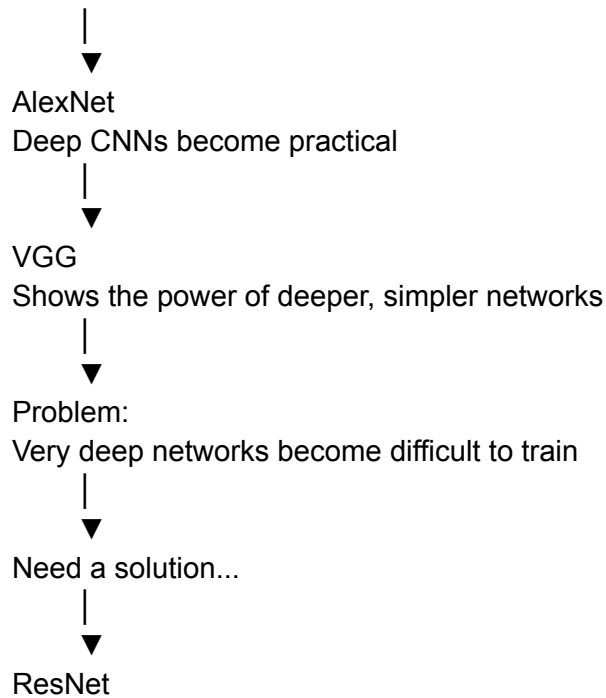
That simple idea became:

Residual Learning.

And it changed everything.

Timeline So Far

Traditional ML



Comparison: AlexNet vs VGG

Feature	AlexNet	VGG
Learnable Layers	8	16 / 19
Filter Sizes	Mixed	Mostly 3×3
Architecture	More varied	Simple and repetitive
Parameters	~60M	~138M
Main Contribution	Started the deep learning revolution	Showed that deeper networks improve performance
Biggest Weakness	Relatively shallow	Very large and difficult to scale much deeper

Why This Matters for Your Passive Sonar Project

Many older sonar papers used:

LOFARGRAM



VGG16



Classification

Later papers often switched to:

LOFARGRAM



ResNet-18



Classification

Why?

Not because VGG is "bad."

But because **ResNet trains deeper networks more effectively, uses parameters more efficiently, and generally achieves better performance.**

Understanding this evolution will help you understand why researchers choose one backbone over another.

Final Thought Before ResNet

Everything you've learned so far leads to one question:

If deeper networks should learn better representations, why do they sometimes perform worse?

The answer is **not** simply "overfitting."

The real breakthrough was discovering **how to make very deep networks train successfully.**

That breakthrough is **ResNet**, introduced in 2015.

Next Chapter: ResNet (The Most Important Architecture)

Take this seriously because **ResNet-18** is one of the most commonly used backbones in passive sonar target classification papers.

We'll spend several lessons on it, including:

1. **Why VGG failed when made deeper.**
2. **The vanishing gradient problem explained intuitively.**
3. **What is a residual (skip) connection?**
4. **How residual learning works.**
5. **Why ResNet can have 18, 34, 50, 101, or even 152 layers.**
6. **Why ResNet-18 is often chosen for sonar classification.**
7. **How ResNet processes a LOFARGRAM from input to prediction.**

In my opinion, **ResNet is the single most important CNN architecture for your project**, so we'll study it in much greater depth than AlexNet or VGG.

Excellent. **Now we are starting the most important architecture of your entire learning journey.**

I'm not exaggerating when I say this:

If you completely understand ResNet, you'll understand nearly 70% of the CNN architectures used in modern computer vision research.

For your **Passive Sonar Based Target Ship Classification** project, this chapter is extremely important because many research papers use **ResNet-18** or **ResNet-34** as their backbone.

We are going to study ResNet like engineers, not like students memorizing definitions.

Deep CNN Architectures

Chapter 3: ResNet (Part 1)

Why Was ResNet Invented?

Before learning ResNet, forget everything you know for a few minutes.

Let's think like researchers.

The Story Begins

It's 2014.

Researchers have VGG.

VGG is amazing.

It achieves great accuracy.

Then someone asks a simple question.

"If 16 layers are good... why not make 50 layers?"

Sounds logical.

More layers...

↓

More learning...

↓

Higher accuracy.

Right?

Everyone believed the answer was **YES**.

So researchers built:

- 20-layer CNN
- 30-layer CNN
- 50-layer CNN
- 100-layer CNN

They expected better performance.

But something shocking happened.

The Surprise

Imagine this graph.

Accuracy

95%
94%
93%
92%
91% *

----->

8 16 20 30 50 Layers

Accuracy improves...

Then suddenly...

Accuracy

95%
94%
93%
92%
91% *
89%
87%
85%
82%
80%

----->

8 16 20 30 50 Layers

The deeper network performs **worse**.

Not just on test data.

Sometimes even on the **training data**.

Wait...

This is strange.

A 50-layer network should at least be able to perform as well as a 20-layer network.

After all,

it has **more learning capacity**.

Why is it failing?

This confused researchers around the world.

Is It Overfitting?

Most beginners immediately answer:

"It's overfitting."

No.

This is one of the biggest misconceptions.

Let's understand why.

Imagine Two Students

Student A studies:

10 chapters.

Scores:

90%.

Student B studies:

The same 10 chapters

PLUS

40 additional chapters.

Scores:

75%.

That doesn't make sense.

The second student could simply ignore the extra chapters and still score 90%.

Right?

Exactly.

Similarly,

a 50-layer network should at least be able to behave like a 20-layer network by learning **identity mappings** (essentially passing information through unchanged).

But it doesn't.

Why?

That was the mystery.

The Real Problem

Researchers discovered something called:

Degradation Problem

This is different from overfitting.

What is the Degradation Problem?

Definition

The **degradation problem** is the phenomenon where **adding more layers to a neural network causes higher training error**, making optimization harder, even though the deeper model should, in theory, be at least as capable as the shallower one.

Notice:

Training Error ↑

Not just Test Error.

This tells us the issue is optimization, not simply memorization.

Think of Building a Bridge

Imagine a bridge with:

10 sections.

Easy to build.

Now imagine:

100 sections.

Every section depends on the previous one.

If one section has a problem,

the whole bridge becomes harder to complete.

Deep neural networks face a similar challenge.

The Learning Signal

During training,

the prediction travels:

Input

↓

Layer 1

↓

Layer 2

↓

Layer 3

↓

...

↓

Output

This is called:

Forward Propagation.

After the prediction,

the network computes the error.

Now the error travels backward.

Output

↓

Layer 100

↓

Layer 99

↓

...

↓

Layer 1

This is called:

Backpropagation.

The Problem

Imagine you're whispering a message through 100 people.

Person 1:

"Ship detected."

Person 2:

Repeats it.

Person 3:

Repeats it.

...

Person 100 hears:

"Something... detected... maybe..."

The message becomes weak.

Similarly,
during backpropagation,
the learning signal (gradient) can become very small as it moves through many layers.
Earlier layers then receive little information about how to improve.
This is closely related to the **vanishing gradient problem**.

Why Does This Matter?

Remember:

The earliest layers learn:

- Edges
- Lines
- Basic frequency patterns

If those layers stop learning,
everything built on top of them suffers.

The Researchers' Thought Process

Instead of asking:

"How do we make better convolution filters?"

They asked:

"How do we make information travel through a very deep network more easily?"

That question led to one of the greatest ideas in deep learning history.

Imagine Climbing a Mountain

Suppose you want to reach the top.

Path 1:

Ground



Step



Step



Step



Mountain Top

Now imagine the path becomes blocked halfway.

You're stuck.

What if someone builds a shortcut?

Ground



Step



Shortcut



Mountain Top

Now reaching the top becomes easier.

This is the basic intuition behind **skip connections**.

The Big Idea

Instead of forcing information to pass through **every** layer,

why not allow it to **skip** some layers when useful?

This became the foundation of **Residual Learning**.

Why Is This So Brilliant?

Imagine one convolution block learns something useful.

Great.

But what if another block **doesn't improve** the representation?

Without ResNet:

The signal must still pass through that block.

With ResNet:

The network has a direct path to carry the original information forward.

The block only needs to learn what is **different** or **additional**, not recreate everything from scratch.

This makes optimization much easier.

The Name "Residual"

Students often ask:

Why is it called **Residual Network**?

Because the network is encouraged to learn the **residual**—the difference between the desired output and the input—rather than learning the entire mapping from scratch.

We'll build an intuitive understanding of this in the next lesson.

Timeline

Traditional CNN

↓

AlexNet

↓

VGG

↓

Problem:

Very Deep Networks Become Hard to Train

↓

Researchers Ask:

How can we help information flow?

↓

ResNet

Summary

Concept	Meaning
VGG showed deeper is better	Up to a point

Very deep networks	Became difficult to optimize
Degradation Problem	Training error increases as depth increases
Not the same as overfitting	It affects training itself
Key Insight	Improve information and gradient flow through the network
Solution	Residual (skip) connections

A Question to Think About

Imagine a 100-layer CNN.

If layers 40–45 don't learn anything useful...

Should the input be **forced** to pass through them?

Or should the network have a way to **bypass** them when appropriate?

That single question changed deep learning.

Next Lesson (ResNet Part 2) — The Most Important Lesson in This Entire Course

In the next lesson, we'll study:

- What exactly is a Skip Connection?
- What is Residual Learning?
- Why does ResNet learn the "difference" instead of the whole mapping?
- Why is learning the residual easier?
- How does a Residual Block work internally?

Once you understand that lesson, you'll understand the core idea behind ResNet, and many modern architectures will suddenly make much more sense.

Excellent. This is **the single most important lesson** in the entire CNN course.

I want you to forget formulas for now.

Today, I want you to understand **the idea** behind ResNet.

When you truly understand this lesson, you'll realize:

ResNet is actually a very simple idea.

The brilliance is not in its complexity.

The brilliance is in asking the **right question**.

Deep CNN Architectures

Chapter 3: ResNet (Part 2)

Residual Learning & Skip Connections

The Problem We Left Last Time

Imagine we have a deep CNN.

Input

↓

Layer 1

↓

Layer 2

↓

Layer 3

↓

Layer 4



Layer 5



Output

Everything must pass through every layer.

Now suppose:

Layer 3 doesn't learn anything useful.

Question:

Should the input still be forced to pass through Layer 3?

Researchers said:

No.

Imagine a Road Journey

Suppose you're traveling from Hyderabad to Visakhapatnam.

Normal route:

Hyderabad



Town A



Town B



Town C



Vizag

Now imagine Town B has:

- Traffic
- Road construction
- Accidents

Should everyone be forced through Town B?

No.

Build a flyover.

Hyderabad



Town A



Flyover



Town C



Vizag

Traffic becomes smoother.

This flyover is exactly like a **Skip Connection**.

What is a Skip Connection?

Definition

A **Skip Connection** (also called a **Shortcut Connection**) is a direct path that allows information to bypass one or more layers and be added to the output of those layers.

Instead of only following:

Input

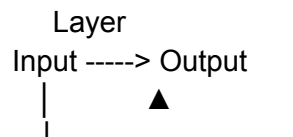


Layer



Output

ResNet creates:



The bottom path is the **shortcut**.

Why Is It Called a Skip Connection?

Because the input literally **skips** some layers.

Imagine a student.

Normal method:

School



College



Job

Shortcut:

Already experienced.

↓

Direct interview.

↓

Job.

The shortcut doesn't eliminate learning.

It simply provides an alternative path.

Let's Build a Residual Block

A Residual Block is the building block of ResNet.

Imagine the input is called:

x

The input enters two paths.

Path 1 (Normal Path)

x

↓

Conv

↓

ReLU

↓

Conv

↓

Output

Let's call this output:

$F(x)$

This is the transformation learned by the convolutional layers.

Path 2 (Shortcut Path)

The shortcut does something surprising.

It simply carries the input forward.

x

↓

(No computation)

↓

Output

No convolution.

No ReLU.

Just the original input.

Then Comes the Magic

At the end of the block, the two paths are combined.

The output becomes:

Output = $F(x) + x$

This is the **core equation of ResNet**.

Don't worry about memorizing it yet.

Let's understand what it means.

What Does This Equation Mean?

Suppose:

Input:

x = Original Information

The convolutional layers learn:

$F(x)$

which represents **new information** or **corrections**.

Instead of throwing away the original input,

ResNet says:

"Keep the original information **and** add whatever useful features the new layers learned."

This preserves information while allowing refinement.

Real-Life Example

Imagine you're writing notes.

Original notes:

Chapter 1 Notes

Your friend improves them by adding:

- Better examples
- Missing points
- Diagrams

Your final notes are:

Original Notes

+

Friend's Improvements

You don't rewrite everything.

You simply **add improvements**.

That is the philosophy of ResNet.

Why Is Learning the Residual Easier?

This is the most important question.

Imagine I ask you:

Write the entire textbook again.

Hard.

Now I ask:

Read the textbook.

Only write the corrections.

Much easier.

ResNet asks each block to learn only what is **missing**.

This "missing part" is called the **residual**.

Sonar Analogy

Suppose your input LOFARGRAM already contains clear engine harmonics.

Without ResNet:

Every new layer has to relearn all the useful information.

With ResNet:

The original harmonic information can flow directly through the shortcut.

The convolutional layers only need to learn **additional patterns**, such as:

- Cavitation noise.
- Fine engine modulation.
- Distinguishing features between ship types.

The network builds upon existing information instead of recreating it.

What Happens If the Convolution Learns Nothing Useful?

Suppose:

$$F(x) = 0$$

Then:

$$\text{Output} = F(x) + x$$

$$\text{Output} = 0 + x$$

$$\text{Output} = x$$

The block simply behaves like an **identity mapping**.

This is a brilliant property.

Even if a block doesn't improve the representation,

it doesn't necessarily make it worse.

Compare With a Normal CNN

Traditional CNN

Input

↓

Conv

↓

Conv

↓

Output

If the convolutional layers learn poorly,
the information is degraded.

ResNet

Input

↓

Conv

↓

Conv

↓

+

↑

Shortcut

Even if the convolutions struggle,
the original information still has a direct path.

Think Like Water

Imagine water flowing through pipes.

Normal CNN:

Pipe

↓

Pipe

↓

Pipe

↓

Pipe

If one pipe is blocked,
water struggles to reach the end.

ResNet:

Pipe

↓

Pipe

↓

Pipe

↓

Pipe

AND

Shortcut Pipe

If the main path becomes difficult,
the shortcut still carries information.

This is why both the forward information and the backward gradients can flow more easily.

Why Does This Help Backpropagation?

Remember backpropagation?

The learning signal travels from the output back to the first layer.

In a very deep traditional CNN, the gradient can become weak.

With shortcut connections, there is a more direct path for the gradient to travel.

This makes optimization of deep networks much easier.

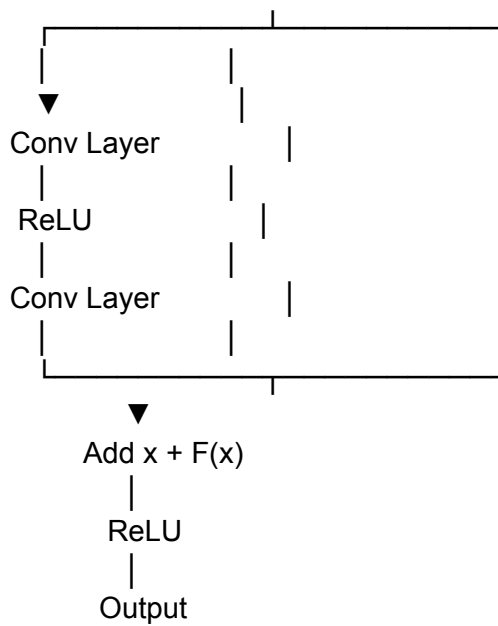
Notice:

The shortcut **helps** gradient flow.

It doesn't magically eliminate every optimization challenge, but it greatly reduces the problem.

Complete Residual Block

Input (x)
|



This small block is repeated again and again to build ResNet-18, ResNet-34, ResNet-50, and deeper variants.

Why This Changed Deep Learning

Before ResNet:

Researchers thought:

"Deeper networks are difficult to train."

After ResNet:

Researchers successfully trained networks with:

- 50 layers.
- 101 layers.
- 152 layers.

Later work even explored much deeper networks.

That was almost impossible with earlier architectures.

Summary

Concept	Meaning
Skip Connection	Direct path that bypasses layers
Shortcut	Carries the original input forward
Residual	The additional information the block learns
Output	Original input + learned improvements
Biggest Advantage	Better information flow and easier optimization of deep networks

One Thought Experiment

Imagine you inherit a perfectly written report.

Would you:

Option A: Throw it away and rewrite every page.

Or

Option B: Keep the report and only add the improvements it needs.

ResNet chooses **Option B**.

That simple idea transformed deep learning.



Homework (Don't Skip This)

Without looking back, explain these three questions in your own words:

1. Why did VGG fail when it became very deep?
2. What is a Skip Connection?
3. Why is learning **only the difference (residual)** easier than learning the entire mapping?

If you can answer these comfortably, you've truly understood the heart of ResNet.

Next Lesson (ResNet Part 3)

In the next lesson, we'll move from the concept to the actual architecture and answer:

- Why are there **ResNet-18, ResNet-34, ResNet-50, ResNet-101, and ResNet-152**?
- What does the number (18, 50, 101...) actually mean?
- What is the difference between a **Basic Block** and a **Bottleneck Block**?
- Why does ResNet-50 use bottleneck blocks?
- Which ResNet version is best for **passive sonar target classification**, and why is **ResNet-18** so commonly used in research?

]Excellent. You're now entering the level where you'll understand **why there are so many ResNet versions** and why almost every sonar paper mentions **ResNet-18** or **ResNet-50**.

This lesson is one of the most important for your project.

Deep CNN Architectures

Chapter 3: ResNet (Part 3)

Understanding ResNet-18, ResNet-34, ResNet-50, ResNet-101 & ResNet-152

First Question

Open any deep learning research paper and you'll see names like:

- ResNet-18

- ResNet-34
- ResNet-50
- ResNet-101
- ResNet-152

Students often think:

"These must be completely different networks."

They are not.

They all belong to the **ResNet family**.

The main difference is **how many layers (or residual blocks) they contain**.

Think of them like different models of the same car.

Real-Life Analogy

Imagine buying a laptop.

You have:

Laptop Series

↓

i3

↓

i5

↓

i7

↓

i9

All are laptops.

The difference is mainly in **power and complexity**.

Similarly:

ResNet Family

↓

ResNet-18

↓

ResNet-34

↓

ResNet-50

↓

ResNet-101

↓

ResNet-152

All use the **same core idea: Residual Learning**.

What Does the Number Mean?

The number roughly represents the **number of learnable layers** in the network.

Examples:

Model	Learnable Layers
ResNet-18	18
ResNet-34	34
ResNet-50	50

ResNet-10 101
1

ResNet-15 152
2

Notice something.

The architecture becomes deeper.

Why Make the Network Deeper?

Imagine identifying ships.

With a small network,

the model may only recognize:

- Engine harmonics
- Tonal lines

A deeper network can also learn:

- Complex engine modulation
- Cavitation patterns
- Higher-level acoustic signatures

More layers generally mean **greater representational power**.

But deeper isn't always better—you need enough data and computation to benefit from the extra depth.

How is ResNet Built?

Instead of thinking in terms of individual layers,

think in terms of **Residual Blocks**.

A Residual Block is the building block of the network.

Imagine LEGO.

One LEGO brick:

[Residual Block]

Now build a wall.

[Block]

↓

[Block]

↓

[Block]

↓

[Block]

The more blocks,

the deeper the network.

ResNet-18

ResNet-18 contains fewer residual blocks.

It is:

- Relatively lightweight
- Fast
- Easy to train
- Less computationally expensive

That's why it's extremely popular in research with limited datasets.

ResNet-152

Now imagine:

Many more residual blocks.

The network becomes much deeper.

Advantages:

- Can learn more complex features.

Disadvantages:

- Requires much more computation.
 - Needs more memory.
 - Usually benefits from larger datasets.
-

Think Like a Student

Suppose two students prepare for an exam.

Student A studies:

10 chapters.

Student B studies:

100 chapters.

Who performs better?

You might say:

Student B.

But what if:

- The exam only covers 10 chapters?

Or

- Student B only had one day to study?

Then studying more wasn't necessarily helpful.

The same idea applies to deep networks.

The model depth should match the problem and the available data.

Why ResNet-18 is Popular in Sonar Research

Now let's connect this to your project.

Imagine your dataset.

You are **not** working with ImageNet.

You are working with sonar data, where datasets are often much smaller.

Using ResNet-152 might:

- Overfit more easily.
- Require much longer training.
- Offer little or no improvement.

ResNet-18 often provides an excellent balance between:

- Accuracy.
- Speed.
- Generalization.

That's one reason why it appears so often in passive sonar classification papers.

Two Types of Residual Blocks

This is something many beginners don't know.

Not every ResNet uses the same kind of block.

There are **two main block types**.

Type 1: Basic Block

Used in:

- ResNet-18
- ResNet-34

Structure:

Input

↓

3×3 Conv

↓

ReLU

↓

3×3 Conv

↓

Add Shortcut

↓

Output

Simple.

Easy to understand.

Type 2: Bottleneck Block

Used in:

- ResNet-50
- ResNet-101
- ResNet-152

Structure:

Input

↓

1×1 Conv

↓

3×3 Conv

↓

1×1 Conv

↓

Add Shortcut

↓

Output

This looks more complicated.

Why?

Because researchers wanted to make very deep networks more computationally efficient.

We'll study bottleneck blocks in detail in the next lesson.

Why Introduce 1×1 Convolutions?

At first glance, a **1×1 convolution** seems useless.

Students ask:

"How can a filter that only looks at one pixel help?"

Excellent question.

The answer is:

It doesn't look across neighboring pixels.

Instead, it learns to **mix and transform information across the channel dimension**.

It can also reduce or increase the number of channels, making later computations much cheaper.

This is one of the clever ideas behind deeper ResNets.

We'll cover it thoroughly next.

Comparing the Models

Model	Block Type	Advantages	Trade-offs
ResNet-18	Basic Block	Fast, lightweight	Lower capacity
ResNet-34	Basic Block	More expressive than 18	More computation
ResNet-50	Bottleneck	Strong accuracy, efficient for its depth	More complex
ResNet-101	Bottleneck	Very powerful	Heavy computation
ResNet-152	Bottleneck	Extremely deep	High memory and compute requirements

Which One Should You Use?

Suppose you're classifying:

- Dogs.
- Cats.
- Cars.

With millions of images.

A deeper model like ResNet-50 or ResNet-101 might make sense.

Now consider your passive sonar project.

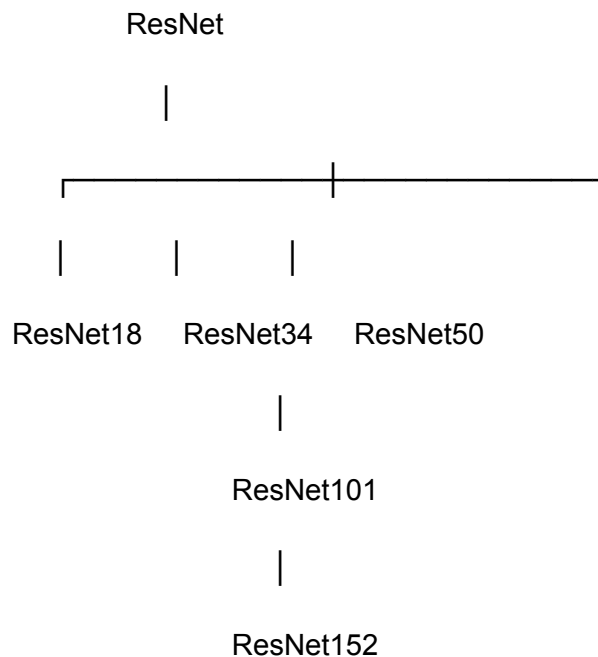
Typical datasets are much smaller.

In many cases:

ResNet-18 or ResNet-34 is a very sensible starting point because they provide strong performance without requiring enormous amounts of data or computation.

Of course, the best choice depends on your dataset size, hardware, and performance goals.

Visualizing the Family



The concept is the same.

Only the depth and block design change.

How Does a LOFARGRAM Flow Through ResNet?

Imagine your input.

LOFARGRAM

↓

Conv Layer

↓

Residual Block

↓

Residual Block

↓

Residual Block

↓

...

↓

Classification

Each block gradually improves the learned representation.

Instead of relearning everything,

each block asks:

"What useful information should I add to what I already know?"

That's the heart of residual learning.

Summary

Concept	Key Idea
ResNet Family	Same core architecture, different depths
ResNet-18	Lightweight, Basic Blocks
ResNet-34	Deeper Basic Blocks
ResNet-50+	Uses Bottleneck Blocks
Deeper Networks	More capacity but higher computational cost
Sonar Applications	ResNet-18 is a common and practical choice for many datasets

One Thought Before the Next Lesson

Today we saw this block:

Input

↓

1×1 Conv

↓

3×3 Conv

↓

1×1 Conv

↓

Output

It probably looks strange.

You may be wondering:

"Why would anyone use a 1×1 convolution?"

That question led to another important breakthrough in CNN design.

Next Lesson (ResNet Part 4)

In the next lesson, we'll go deep into the **Bottleneck Block**, where you'll learn:

- What exactly is a **1×1 convolution**?
- How can it reduce computation without throwing away useful information?
- Why does ResNet-50 use bottleneck blocks instead of basic blocks?
- How does the bottleneck design make very deep networks practical?
- Why do modern architectures like **DenseNet**, **EfficientNet**, and many others also use **1×1 convolutions**?

By the end of that lesson, you'll understand one of the smartest design ideas in modern CNNs.

Next

Excellent! This lesson is one of the **deepest concepts in CNN architecture**.

When students first see a **1×1 convolution**, almost everyone says:

"This makes no sense."

Because they think:

"A convolution should look at neighboring pixels."

A 1×1 filter doesn't even look at neighbors.

So...

Why does it exist?

Today we'll answer that.

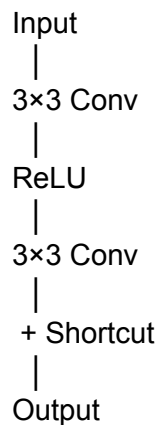
Deep CNN Architectures

Chapter 3: ResNet (Part 4)

The Bottleneck Block & the Mystery of 1×1 Convolutions

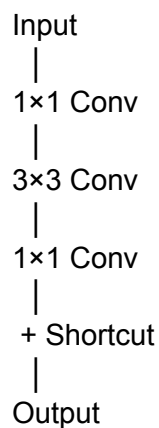
First, Let's Recall the Basic Block

In ResNet-18, one block looks like:



Simple.

Now look at ResNet-50.



Students immediately ask:

"Why add two extra layers?"

Good question.

Imagine a Big Library

Suppose you enter a library with **1000 books**.

You only need books about:

- Ships
- Ocean Acoustics
- Signal Processing

Would you carry all 1000 books everywhere?

No.

You would first select only the useful books.

Then study them.

Then organize your final notes.

That is exactly what a Bottleneck Block does.

The Secret of Channels

Remember from previous lessons:

After several convolution layers, your feature map may look like:

$56 \times 56 \times 256$

Let's understand this carefully.

- Height = 56
- Width = 56
- Channels = 256

Those **256 channels** represent **256 different learned features**.

Think of them as 256 experts.

Expert 1:

Looks for harmonic lines.

Expert 2:

Looks for broadband noise.

Expert 3:

Looks for engine modulation.

...

Expert 256:

Looks for another acoustic pattern.

The Problem

Now suppose we apply a **3×3 convolution** directly.

Every filter has to process **all 256 channels**.

That is computationally expensive.

Imagine asking one person to interview **256 experts** before making a decision.

Slow.

The Brilliant Idea

Instead of using all 256 channels,

what if we first reduce them?

Example:

$56 \times 56 \times 256$

↓

1×1 Conv

↓

$56 \times 56 \times 64$

Notice something.

Height:

Still 56.

Width:

Still 56.

Only the channels changed:

256

↓

64

The network has compressed the information.

Wait...

How Can 1×1 Convolution Reduce Channels?

Excellent question.

This is where most tutorials stop.

Let's understand it.

Think About Mixing Colors

Imagine you have:

- Red
- Green
- Blue

You mix them.

The result is a new color.

Did you look at neighboring pixels?

No.

You only mixed information at the **same pixel location**.

A **1×1 convolution** does the same thing.

It combines information **across channels**, not across space.

Example

Suppose one pixel has values:

Channel 1 = 10

Channel 2 = 7

Channel 3 = 15

Channel 4 = 3

A 1×1 filter learns weights like:

0.2

0.5

-0.3

0.8

It combines those channel values into a new feature.

The filter is learning:

"How important is each channel?"

This is why people often describe 1×1 convolutions as **channel mixers** or **channel projections**.

Three Stages of a Bottleneck Block

Now let's understand the complete design.

Stage 1: Reduce

Input:

$56 \times 56 \times 256$

↓

1×1 Conv

↓

$56 \times 56 \times 64$

Purpose:

Reduce the number of channels.

This saves computation.

Stage 2: Learn

Now apply:

3×3 Conv

But notice:

Instead of processing **256 channels**,

it only processes **64 channels**.

Much cheaper.

This is where the main spatial feature learning happens.

Stage 3: Expand

Finally,

another:

1×1 Conv

restores the channel dimension.

Example:

$56 \times 56 \times 64$

↓

$56 \times 56 \times 256$

Now the output has the expected shape again, so it can be combined with the shortcut path.

Real-Life Analogy

Imagine writing a report.

Step 1:

Remove unnecessary details.

↓

Step 2:

Analyze the important information.



Step 3:

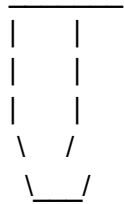
Write the complete report again.

You don't analyze everything.

You analyze the essential parts.

Why Is This Called a Bottleneck?

Think of a water bottle.



The neck is narrow.

Everything passes through that narrow region.

Similarly:

256 Channels



64 Channels



256 Channels

The information is squeezed into a smaller representation before expanding again.

That's why it's called a **Bottleneck Block**.

Why Doesn't Information Get Lost?

Students often worry:

"If we reduce 256 channels to 64, aren't we throwing away information?"

A reasonable concern.

The answer is:

The network **learns** which channel combinations are most useful.

If reducing too aggressively hurts performance, training will adjust the weights to preserve important information.

So while compression can discard some redundancy, the goal is to keep the information that matters most for the task.

Sonar Example

Imagine your LOFARGRAM produces channels like:

- Engine harmonics
- Propeller tones
- Broadband noise
- Cavitation
- Ambient ocean noise
- Machinery modulation
- Other learned patterns

Not every channel is equally important for every decision.

The 1×1 convolution learns how to combine and emphasize the most informative channel patterns before the more expensive 3×3 convolution.

Why Is Bottleneck So Powerful?

Without a bottleneck:

3×3 Conv

↓

Huge Computation

With a bottleneck:

Reduce Channels

↓

3×3 Conv

↓

Expand Channels

The expensive spatial convolution operates on fewer channels, making the network much more computationally efficient.

This is one reason ResNet-50 can be much deeper than ResNet-18 while remaining practical.

Basic Block vs Bottleneck Block

Basic Block	Bottleneck Block
$3 \times 3 \rightarrow 3 \times 3$	$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$
Used in ResNet-18 & 34	Used in ResNet-50, 101 & 152
Simpler	More computationally efficient for deep networks
Good for shallower models	Better suited for very deep models

Why This Matters Beyond ResNet

The idea of **1×1 convolutions** didn't stop with ResNet.

You'll encounter them in many important architectures, including:

- **GoogLeNet (Inception)**
- **DenseNet**
- **MobileNet**
- **EfficientNet**

So understanding this concept now will make those architectures much easier to learn.

Key Takeaways

1. **1×1 convolutions do not look across neighboring pixels.**
 2. **They learn relationships between channels.**
 3. **They can reduce or increase the number of channels.**
 4. **Bottleneck blocks reduce computation while preserving strong performance.**
 5. **That's why ResNet-50 and deeper models use bottleneck blocks instead of basic blocks.**
-

Think Like an Engineer

Suppose you're building a passive sonar classifier.

Your feature maps have **512 channels**.

Would you:

- Run expensive 3×3 convolutions on all 512 channels?

Or

- First compress the channels to a smaller, informative representation, perform the expensive operation there, and then expand them again?

Modern CNN designers usually choose the second approach because it provides a much better balance between computational cost and representational power.

Next Lesson (ResNet Part 5 – Final)

In the final ResNet lesson, we'll connect everything you've learned and answer:

- How does a complete LOFARGRAM travel through ResNet-18?
- What happens at every stage of the network?
- Why is ResNet-18 one of the most popular choices for passive sonar target classification?
- When should you choose ResNet-18 vs ResNet-50?
- How is ResNet used with transfer learning in real research papers?

That lesson will tie the entire ResNet chapter directly to **your passive sonar project** and prepare you for understanding modern sonar classification research.

Ok next

Excellent. This is the **final chapter on ResNet**, where we'll connect everything directly to **your passive sonar target classification project**.

After this lesson, you won't just know ResNet—you'll understand **how researchers actually use it** in real-world sonar systems.

Deep CNN Architectures

Chapter 3: ResNet (Part 5 – Final)

ResNet in Passive Sonar Target Classification

The Big Picture

Let's assume your goal is:

Binary Classification

Input: Passive sonar recording

Output: Ship Present  or Ship Absent 

At first glance, this seems impossible.

How can a neural network understand an audio signal?

The answer is:

It doesn't work directly on raw audio in most traditional CNN pipelines.

Instead, we first convert the audio into a visual representation.

Step 1: Passive Sonar Signal

Imagine a hydrophone records sound.

The raw waveform looks like this:

Time →



This waveform is difficult for a CNN to process directly.

So we transform it.

Step 2: Signal Processing

You already learned these techniques:

Raw Audio

↓

FFT

↓

STFT

↓

Spectrogram

↓

Mel Spectrogram

or

LOFARGRAM

For passive sonar, **LOFARGRAM** is one of the most common choices because it highlights frequency components over time that are useful for detecting ship signatures.

Now the signal becomes an image-like representation.

Step 3: Feed the LOFARGRAM to ResNet

Imagine your LOFARGRAM:

```
+-----+
| /////      |
| ///===== |
| ===////     |
| ////===     |
+-----+
```

To a human,

this looks like a heatmap.

To ResNet,

it's simply an image.

What Does the First Layer Learn?

The first convolution sees only small regions.

It learns simple patterns like:

- Horizontal lines
- Vertical lines
- Bright frequency bands
- Small edges

In sonar, these may correspond to:

- Narrow tonal lines
- Local changes in energy
- Short-duration events

These are the building blocks.

Middle Layers

The next residual blocks combine those simple patterns.

Instead of individual frequency lines,

they begin recognizing:

- Harmonic groups
- Repeating tonal structures
- Frequency trajectories

These are much more informative than isolated pixels.

Deep Layers

Now ResNet starts recognizing high-level acoustic patterns.

For example:

- Engine acoustic signatures
- Propeller-related harmonics
- Stable frequency structures associated with ships

At this point, the network isn't "seeing lines" anymore.

It's learning abstract representations that help distinguish:

Ship Present vs Ship Absent.

Final Classification

After many residual blocks:

LOFARGRAM

↓

Residual Blocks

↓

Global Average Pooling

↓

Fully Connected Layer

↓

Softmax / Sigmoid

↓

Prediction

For your binary problem, the final output might be:

Ship Present = 0.97

Ship Absent = 0.03

The network predicts **Ship Present** because it has the higher probability.

Why Global Average Pooling?

Earlier architectures like AlexNet and VGG used very large **fully connected layers**, which introduced millions of parameters.

ResNet instead uses **Global Average Pooling (GAP)** before the final classifier.

Imagine the last feature map:

$$7 \times 7 \times 512$$

Instead of flattening everything,

GAP computes the **average value of each channel**.

So:

$$7 \times 7 \times 512$$

↓

512 numbers

Each number summarizes one feature channel.

Advantages:

- Far fewer parameters.
- Lower risk of overfitting.
- Better generalization.
- Less memory usage.

This is one reason ResNet is much more efficient than VGG.

Why is ResNet-18 So Popular?

Suppose you have:

- 2,000 sonar recordings.

Would ResNet-152 be a good choice?

Probably not.

It has very high capacity and may overfit if the dataset is too small.

ResNet-18 often works well because:

- It's easier to train.
- It's computationally efficient.
- It generalizes well on many medium-sized datasets.
- It has significantly fewer parameters than deeper ResNets.

This is why you'll frequently see **ResNet-18** in passive sonar research.

Transfer Learning

This is one of the most important ideas for your project.

Imagine you train ResNet from scratch.

The network must learn:

- Basic edges.
- Frequency patterns.
- Intermediate structures.
- High-level representations.

That requires a lot of data.

Instead, researchers often start with a **ResNet pretrained on ImageNet**.

You might wonder:

"ImageNet contains natural images, not sonar. Why would this help?"

Because the **early layers** learn very general features like edges, gradients, and simple textures.

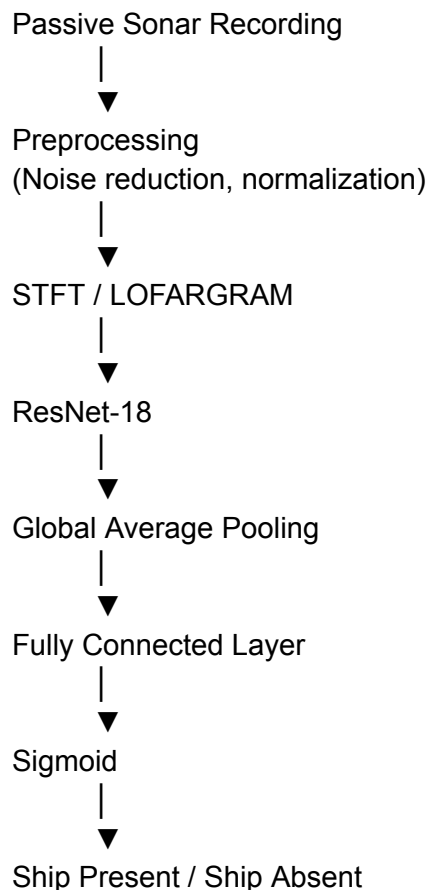
Those low-level feature extraction capabilities can still be useful when fine-tuned on sonar images.

Then researchers replace the final classification layer and train the network on their sonar dataset.

This process is called **Transfer Learning**.

Your Complete Pipeline

Now you can finally understand an end-to-end passive sonar classifier.



This is very close to the architecture used in many research papers.

Why ResNet Fits Your Project

Your project involves recognizing subtle acoustic patterns.

ResNet helps because:

- Residual connections improve optimization.
 - It learns hierarchical features automatically.
 - It is robust enough for complex acoustic images.
 - It works well with transfer learning.
 - ResNet-18 offers an excellent balance of accuracy and efficiency.
-

Comparing AlexNet, VGG, and ResNet

Feature	AlexNet	VGG	ResNet
Main Idea	First successful deep CNN	Showed the power of depth	Solved the degradation problem with residual learning
Learnable Layers	8	16 / 19	18–152+
Residual Connections	✗	✗	✓
Global Average Pooling	✗	✗	✓
Large Fully Connected Layers	✓	✓	Minimal
Common in Modern Sonar Research	Rare	Sometimes	Very common

A Researcher's Thought Process

When reading a sonar paper that says:

"We used ResNet-18."

You should now ask:

- Why ResNet-18 instead of VGG?
- How large was their dataset?
- Did they use transfer learning?
- What input representation did they use? (LOFARGRAM, spectrogram, Mel spectrogram?)
- Why was ResNet-18 sufficient?

These are the kinds of questions that distinguish someone who understands the field from someone who only knows the model names.

What You've Learned So Far

You now understand:

Digital Audio Processing

- FFT
- STFT
- Spectrogram
- Mel Spectrogram
- MFCC
- LOFARGRAM

Machine Learning

- Features
- Labels
- Training
- Validation
- Testing
- Loss
- Gradient Descent
- Backpropagation
- Overfitting
- Regularization

Deep Learning

- Neurons

- Activation Functions
- CNNs
- Convolution
- Pooling
- Feature Maps
- ReLU
- Dropout
- Batch Normalization

CNN Architectures

- AlexNet
- VGG
- ResNet

You've built a solid foundation.

Where Do We Go Next?

Now we move from **classic CNNs** to **modern architectures**.

The order I recommend is:

1. **DenseNet** (understand dense connections and feature reuse)
 2. **Inception / GoogLeNet** (multi-scale feature extraction)
 3. **EfficientNet** (compound scaling; very relevant to your project)
 4. **MobileNet** (lightweight models)
 5. **Vision Transformers (ViT)** (modern alternative to CNNs)
-

★ My recommendation

Since your project is **Passive Sonar Based Target Ship Classification**, the next architecture should be **DenseNet**.

Why?

Because many sonar classification papers compare:

- ResNet

- DenseNet
- EfficientNet

DenseNet introduces a different idea from ResNet:

Instead of connecting only the input to the output of a block, what if every layer could directly access the outputs of all previous layers?

That simple idea led to another major breakthrough in deep learning.

From here onward, you'll start recognizing the design philosophy behind almost every modern CNN architecture.

Comparison

ok ok from this i had an request so i want an clear comparisions of different pretrained models available in the CNN give me indepth comparision also give me a clear comparision table

Excellent request. This is actually something every deep learning researcher should know.

Most people simply write:

"We used ResNet-18 because it gives good accuracy."

A researcher should be able to answer:

- **Why not VGG?**
- **Why not DenseNet?**
- **Why not EfficientNet?**
- **Why not MobileNet?**
- **Why not Vision Transformer (ViT)?**

If your project review panel asks you this question, you should be able to defend your choice.

Complete Comparison of Popular Pretrained CNN Models

We'll compare the most widely used ImageNet-pretrained models:

1. AlexNet (2012)
2. VGG16 / VGG19 (2014)

3. GoogLeNet / Inception V1 (2014)
 4. ResNet Family (2015)
 5. DenseNet (2017)
 6. MobileNet Family
 7. EfficientNet Family
 8. ConvNeXt (2022)
 9. Vision Transformer (ViT) (*not a CNN, but important for comparison*)
-

1. AlexNet (2012)

Why was it revolutionary?

Before AlexNet, most computer vision relied on handcrafted features.

AlexNet demonstrated that a deep CNN trained on enough data could automatically learn useful features.

It won the 2012 ImageNet competition by a large margin.

Architecture

Input Image

↓

5 Convolution Layers

↓

3 Fully Connected Layers

↓

Output

Characteristics

Property	Value
Year	2012
Layers	8
Parameters	~61 Million
Input Size	224×224
Speed	Medium
Memory	High

Advantages

- ✓ Simple architecture
 - ✓ Historically important
-

Disadvantages

- ✗ Very old
 - ✗ Large number of parameters
 - ✗ Lower accuracy than modern models
-

Use Today?

Mostly for educational purposes.

Rarely used in research.

2. VGG16 / VGG19

Main Idea

Instead of using large filters,

VGG repeatedly uses:

3×3 Convolution

Many times.

Architecture

Conv

↓

Conv

↓

Pooling

↓

Conv

↓

Conv

↓

Pooling

Very repetitive.

Very easy to understand.

Characteristics

Property	VGG16
Layers	16
Parameters	~138 Million
Accuracy	Good
Speed	Slow
Memory	Very High

Advantages

- ✓ Easy to understand
 - ✓ Strong feature extractor
-

Disadvantages

- ✗ Extremely large model
 - ✗ Slow training
 - ✗ Large memory consumption
-

Research Usage

Still widely used for:

- Feature extraction
- Medical imaging
- Transfer learning

But less common for new state-of-the-art systems.

3. GoogLeNet (Inception)

Google asked:

"Why choose one filter size?"

Instead,

they used multiple filter sizes simultaneously.

1×1

3×3

5×5

Pooling

↓

Concatenate

This is called the **Inception Module**.

Advantages

- ✓ More efficient than VGG
 - ✓ Better accuracy
 - ✓ Fewer parameters
-

Disadvantages

- ✗ More complex architecture
 - ✗ Harder to modify
-

Parameters

~7 Million

Much smaller than VGG.

4. ResNet Family

One of the most influential CNN architectures.

Problem:

Very deep networks became difficult to train.

Solution:

Residual Learning.

Instead of learning:

Output = $H(x)$

ResNet learns:

Output = $x + F(x)$

This **skip connection** helps gradients flow through very deep networks.

Variants

Model	Layers
ResNet18	18
ResNet34	34
ResNet50	50
ResNet101	101
1	

Advantages

- ✓ Very stable
 - ✓ Easy to fine-tune
 - ✓ Excellent transfer learning
 - ✓ Widely used in research
-

Disadvantages

- ✗ Larger variants require more computation
-

Your Project

This is why I recommended **ResNet-18**:

- Small dataset friendly
 - Stable training
 - Fast
 - Strong baseline
 - Excellent for transfer learning
-

5. DenseNet

DenseNet introduces **dense connectivity**.

Instead of connecting only adjacent layers,

every layer receives outputs from all previous layers.

Example:

Layer1



Layer2



Layer3



Layer4

In reality:

Every layer connects to every later layer within a dense block.

Advantages

- ✓ Excellent feature reuse
 - ✓ Strong gradient flow
 - ✓ High accuracy with fewer parameters
-

Disadvantages

- ✗ More memory usage due to many feature maps
 - ✗ Slightly slower
-

Research Usage

Very common in:

- Medical imaging
 - Small datasets
 - Scientific imaging
-

6. MobileNet

Designed for:

Phones

Embedded Systems

Edge AI

Instead of standard convolution,

it uses:

Depthwise Separable Convolution

This greatly reduces computation.

Advantages

- ✓ Extremely fast
 - ✓ Very small
 - ✓ Low power consumption
-

Disadvantages

✗ Slightly lower accuracy than larger CNNs

Perfect for:

Real-time deployment.

7. EfficientNet

Google asked:

Instead of increasing:

Only depth

OR

Only width

OR

Only image size

Why not scale all three together?

This idea is called:

Compound Scaling

EfficientNet variants:

Model

B0

B1

B2

B3

B4

B5

B6

B7

Advantages

- ✓ Very high accuracy
 - ✓ Excellent parameter efficiency
 - ✓ Modern architecture
-

Disadvantages

- ✗ More computationally demanding than ResNet-18
 - ✗ Training can be more sensitive to hyperparameters
-

Research Usage

One of the strongest CNN families for image classification.

8. ConvNeXt

Researchers asked:

"Can we modernize CNNs using ideas inspired by Transformers?"

Result:

ConvNeXt.

It keeps convolutions but incorporates several design improvements.

Advantages

✓ State-of-the-art CNN performance

✓ Strong transfer learning

Disadvantages

✗ Larger computational requirements

✗ Less beginner-friendly

9. Vision Transformer (ViT)

Important:

ViT is **not** a CNN.

Instead of convolutions,

it splits the image into patches.

Example:

Image

↓

Patch

Patch

Patch



Transformer

Advantages

- ✓ Excellent on large datasets
- ✓ State-of-the-art performance in many tasks

Disadvantages

- ✗ Data hungry
- ✗ Computationally expensive
- ✗ Often benefits from very large pretraining datasets

Complete Comparison Table

Model	Ye ar	Parame ters (Appro x.)	Speed	Memory	Accuracy	Best For	Beginner Friendly
AlexNet	2012	61M	★★★★	★★	★★	Learning history	★★★★★
VGG16	2014	138M	★★	★	★★★★	Feature extraction	★★★★★
GoogLe Net	2014	7M	★★★★★	★★★★★	★★★★★	Efficient CNNs	★★★★

ResNet18	20 15	11.7M	★★★★★ ★	★★★★★	★★★★★	Small/medium datasets, transfer learning	★★★★★ ★
ResNet50	20 15	25.6M	★★★★★	★★★★	★★★★★ ★	General-purpose research	★★★★★
DenseNet121	20 17	8M	★★★★	★★★★	★★★★★ ★	Medical/scientific imaging	★★★★
MobileNetV2	20 18	3.5M	★★★★★ ★	★★★★★ ★	★★★★★	Mobile & edge devices	★★★★★
EfficientNet-B0	20 19	5.3M	★★★★★	★★★★★	★★★★★ ★	High accuracy with fewer parameters	★★★★★
ConvNeXt-Tiny	20 22	28M	★★★★	★★★★	★★★★★ ★	Modern CNN research	★★★★
ViT-Base*	20 20	86M	★★	★★	★★★★★ ★	Large datasets	★★

*ViT is included for comparison even though it is not a CNN.

Which Model Should You Choose?

If your dataset is small (fewer than ~10,000 images)

✓ ResNet-18

✓ DenseNet-121

These tend to be reliable starting points when combined with transfer learning.

If your dataset is medium (roughly 10,000–100,000 images)

- ✓ ResNet-50
- ✓ EfficientNet-B0/B1

If your dataset is very large

- ✓ EfficientNet variants
- ✓ ConvNeXt
- ✓ ViT (especially if pretrained on large datasets)

Recommendation for Your Passive Sonar Project

Let's consider your project:

- Binary Classification (Ship / No Ship)
- LOFARGRAM images
- Transfer Learning
- Likely limited dataset size
- Research-oriented project

Phase 1 (Baseline)

ResNet-18

Why?

- Easy to train
- Fast experimentation
- Strong transfer learning

- Well understood in the literature
-

Phase 2 (Improved Performance)

EfficientNet-B0

Why?

- Better parameter efficiency
 - Often improves accuracy without a huge increase in model size
-

Phase 3 (Advanced Comparison)

DenseNet-121

Why?

- Excellent feature reuse
 - Frequently performs well on scientific imaging tasks
-

Optional Advanced Experiment

Compare:

- ResNet-18
- EfficientNet-B0
- DenseNet-121

using **exactly the same**:

- Dataset split
- Preprocessing
- LOFARGRAM generation
- Optimizer
- Learning rate
- Epochs
- Evaluation metrics

This creates a fair scientific comparison.

A Researcher's Decision Process

Instead of asking:

"Which model has the highest ImageNet accuracy?"

Ask:

- Does my dataset have enough samples?
- Can my GPU handle the model?
- How important is inference speed?
- Is interpretability important?
- Am I optimizing for publication-quality comparisons or deployment?

Those questions lead to better model choices than simply picking the newest architecture.



What I Recommend We Do Next

Before writing any code, I recommend a dedicated **masterclass** where we study each architecture **one by one**, just as we did for ResNet.

We'll cover:

1. **AlexNet** – why it changed computer vision.
2. **VGG** – why stacks of 3×3 convolutions work.
3. **GoogLeNet/Inception** – multi-scale feature extraction.
4. **ResNet** – residual learning (already learned, but we'll compare it deeply).
5. **DenseNet** – dense connectivity and feature reuse.
6. **MobileNet** – depthwise separable convolutions.
7. **EfficientNet** – compound scaling.
8. **ConvNeXt** – the modern CNN.
9. **Vision Transformer** – how it differs fundamentally from CNNs.

For each model, we'll study:

- Why it was invented.
- The problem it solved.
- Architecture block-by-block.
- Mathematical intuition.
- Advantages and disadvantages.

[illegible]

Main Innovation	Deep CNN + ReLU	Small 3×3 convolutions	Inception modules	Residual (Skip) Connections	Deeper Residual Network	Dense Connectivity	Depth wise Separable Convolution	Compound Scaling	Modernized CNN Design	Self-Attention
Primary Goal	Provide deep learning works	Increase depth	Efficient computation	Train very deep networks	Improve representation	Feature reuse	Mobile inference	Better accuracy with fewer parameters	Compete with Transformers	Replace convolutions
Depth (Layers)	8	16	22	18	50	121	53 (including depth wise layers)	~237 operations	~28 blocks	12 Transformer blocks
Trainable Parameters	~61 Million	~138 Million	~7 Million	~11.7 Million	~25.6 Million	~8 Million	~3.5 Million	~5.3 Million	~28 Million	~86 Million
Approximate FLOPs (224×224)	~0.7 GFLOPs	~15.5 GFLOPs	~1.5 GFLOPs	~1.8 GFLOPs	~4.1 GFLOPs	~2.9 GFLOPs	~0.3 GFLOPs	~0.39 GFLOPs	~4.5 GFLOPs	~17.5 GFLOPs
Model Size	~240 MB	~528 MB	~27 MB	~45 MB	~98 MB	~33 MB	~14 MB	~20 MB	~110 MB	~330 MB
Memory Consumption During	High	Very High	Mode rate	Mode rate	High	High	Very Low	Low	High	Very High

Trainin g

Inferen ce Speed	Mod erat e	Slow	Fast	Fast	Moder ate	Moderat e	Very Fast	Fast	Moderate	Slow
Trainin g Speed	Mod erat e	Slow	Fast	Fast	Moder ate	Moderat e	Fast	Mode rate	Moderate	Slow
Compu tationa l Compl exity	Medi um	Very High	Medi um	Mediu m	High	Medium -High	Very Low	Low	High	Very High
GPU Requir ement	Medi um	High	Medi um	Mediu m	Mediu m-Hig h	Medium	Low	Low	High	Very High
CPU Deploy ment	Poss ible	Diffic ult	Good	Good	Accep table	Accepta ble	Excell ent	Excell ent	Heavy	Poor
Input Resolu tion (Defaul t)	224 ×224	224× 224	224× 224	224× 224	224×2 24	224×22 4	224×2 24	224× 224	224×22 4	224× 224
Featur e Extract ion Quality	Basi c	Excel lent	Very Good	Excell ent	Excell ent	Excellen t	Good	Excell ent	Excelle nt	Excel lent
Gradie nt Flow	Poor in deep er versi ons	Poor	Bette r	Excell ent (Skip Conn ection s)	Excell ent	Excellen t (Dense Connect ions)	Good	Good	Excelle nt	Excel lent

Vanishing Gradient Resistance	Low	Low	Medium	Excellent	Excellent	Excellent	Good	Good	Excellent	Excellent
Feature Reuse	Low	Low	Medium	Medium	Medium	Very High	Medium	Medium	High	High
Parameter Efficiency	Poor	Very Poor	Excellent	Good	Good	Excellent	Excellent	Outstanding	Good	Moderate
Transfer Learning Performance	Fair	Excellent	Very Good	Excellent	Excellent	Excellent	Very Good	Excellent	Excellent	Excellent
Works Well on Small Datasets	No	Sometimes	Yes	Yes	Yes (with transfer learning)	Yes	Yes	Yes	Moderate	Usually requires strong pretraining
Works Well on Large Datasets	Limited	Good	Good	Excellent	Excellent	Excellent	Good	Excellent	Excellent	Outstanding
Overfitting Risk	High	Very High	Moderate	Low	Moderate	Low	Low	Low	Moderate	Moderate

Interpretability (Grad-CAM etc.)	Easy	Easy	Moderate	Excellent	Excellent	Excellent	Good	Good	Good	Different visualization methods
Fine-Tuning Difficulty	Easy	Easy	Moderate	Easy	Easy	Moderate	Easy	Moderate	Moderate	More complex
Robustness to Noise	Low	Moderate	Good	Good	Very Good	Very Good	Good	Very Good	Excellent	Very Good
Deployment on Edge Devices	Poor	Poor	Moderate	Good	Moderate	Moderate	Excellent	Excellent	Moderate	Poor
Typical Applications	Education	Feature extraction	General vision	Medical imaging, remote sensing, industrial AI	General research	Medical imaging, hyperspectral imaging	Mobile apps, IoT	High-performance vision	Modern vision tasks	Large-scale vision and multimodal AI
Major Strength	Historic importance	Strong generic features	Efficient multi-scale learning	Stable and reliable	Excellent balance of depth and performance	Strong feature reuse	Extremely lightweight	Excellent accuracy per parameter	State-of-the-art CNN design	Powerful global context modeling

Major Weakness	Outdated	Huge model size	Complex architecture	Shallower than newer models	Heavier than ResNet18	Memory-intensive during training	Slightly lower accuracy ceiling	More sensitive to tuning	More computationally demanding	Requires large datasets and compute
-----------------------	----------	-----------------	----------------------	-----------------------------	-----------------------	----------------------------------	---------------------------------	--------------------------	--------------------------------	-------------------------------------

Note: FLOPs, parameter counts, and model sizes are approximate and can vary slightly depending on the implementation (e.g., PyTorch vs TensorFlow) and the exact variant.

Suitability for Different Types of Projects

Project Type	Best Models	Reason
Small Dataset (<5k images)	ResNet18, DenseNet121	Transfer learning works well and overfitting is manageable
Medium Dataset (5k–50k)	ResNet50, EfficientNet-B0, DenseNet121	Better feature learning while still trainable
Large Dataset (>100k)	EfficientNet, ConvNeXt, ViT	Can exploit larger datasets effectively
Medical Imaging	DenseNet121, ResNet50	Strong feature reuse and stable gradients
Audio Spectrogram Classification	ResNet18, ResNet50, EfficientNet-B0	Proven performance on spectrogram-like inputs
Mobile Deployment	MobileNetV2, EfficientNet-Lite	Small, fast, energy-efficient
Real-Time Detection	MobileNetV2, ResNet18	Low latency
Scientific Research Baseline	ResNet18, ResNet50	Standard baselines used in many papers
Highest Accuracy (General Vision)	EfficientNet, ConvNeXt, ViT	Strong performance on many benchmarks

Comparison for Your Passive Sonar Project

Criteria	ResNet18	ResNet50	DenseNet 121	EfficientNet -B0	MobileNetV2	ConvNeXt-Tiny
Learns LOFARGRAM patterns well	✓	✓	✓	✓	✓	✓
Transfer Learning Quality	Excellent	Excellent	Excellent	Excellent	Very Good	Excellent
Small Dataset Performance	Excellent	Very Good	Excellent	Very Good	Good	Moderate
Training Time	Fast	Medium	Medium	Medium	Very Fast	Slow
GPU Memory Requirement	Low	Medium	Medium	Low	Very Low	High
Risk of Overfitting	Low	Medium	Low	Low	Low	Medium
Ease of Fine-Tuning	Very Easy	Easy	Moderate	Moderate	Easy	Moderate
Research Popularity	Very High	Very High	High	Very High	High	Growing
Inference Speed	Fast	Medium	Medium	Fast	Very Fast	Medium

Good Baseline for Research	★★★★★	★★★★★	★★★★★	★★★★★	★★★★★	★★★★★
Recommended for Your Project	Yes (Baseline)	Yes (Comparison)	Yes (Comparison)	Yes (Strong Candidate)	Optional	Advanced Experiment

My recommendation for your research

If you want your passive sonar project to be **research-grade**, I would compare at least these three models under identical conditions:

Model	Why Include It?
ResNet18	Standard baseline, fast training, highly cited, ideal for transfer learning
DenseNet121	Tests whether dense feature reuse improves sonar pattern recognition
EfficientNet-B0	Evaluates whether a more parameter-efficient modern CNN improves accuracy

If you have enough computational resources, you can add:

- **ResNet50** (deeper residual network)
- **ConvNeXt-Tiny** (modern CNN)
- **ViT-Base** (Transformer baseline, if you want to compare CNNs vs Transformers)

I can make an even more advanced table.

The table above is about **architecture-level comparison**. Researchers also compare models at a much deeper level, including:

- Layer-by-layer architecture
- Block structure (Residual Block vs Dense Block vs MBConv vs Inverted Residual, etc.)
- Skip connection mechanisms
- Mathematical equations
- Activation functions

- Normalization layers
- Receptive field growth
- Feature map sizes at each stage
- Computational complexity analysis (Big-O intuition)
- Memory access patterns
- FLOPs vs latency trade-offs
- Gradient propagation characteristics
- Transfer learning behavior
- Why each architecture was invented and what problem it solved
- Strengths and weaknesses for **spectrogram/LOFARGRAM** classification specifically

That comparison would span **20–30 pages** and would be at the level of a graduate deep learning course or research handbook. Given your goal of deeply understanding these models for your sonar project, I think it would be worth building that reference step by step.